Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

Lenguaje de Programación II

Guía Práctica — Sesión 9

Formularios transaccionales cabecera–detalle: registro y consulta de ventas

Dato	Detalle	Dato	Detalle
Unidad	II. Frontend SPA empresarial seguro	Sesión	9 · 8 de octubre de 2026
Modalidad	Individual, en laboratorio	Duración	2 horas prácticas
Tecnologías	Angular 22 · TypeScript · Spring Boot 4	Proyecto base	PharmaSoft (sesiones 7 y 8) + PharmaBackend
Docente	Reyna Barreto Benjamin David	Ciclo	IV · Semestre 2026-2

1. Situación de la sesión

La botica ya tiene su catálogo en PharmaSoft: categorías y productos con su stock. Pero las ventas todavía se registran en una libreta y, al cierre del día, alguien suma a mano los importes y descuenta el stock en la hoja de cálculo. Los errores son frecuentes: se venden productos sin stock, se escribe dos veces el mismo producto en una boleta y nadie sabe con certeza cuánto se vendió en la semana.

Una venta no es un registro simple: tiene una **cabecera** (cliente, fecha, total) y un **detalle** con una o varias líneas (producto, cantidad, precio, subtotal), y todas se guardan juntas o no se guarda ninguna. Hoy construirás el formulario que permite armar esa venta, revisarla y confirmarla, y la consulta para encontrarla después.

¿Cómo construyes un formulario que arme una venta de varias líneas, calcule su total y solo la registre cuando el usuario la confirme y los datos sean válidos?

Reto 01 (actividad práctica, en clase): implementar el registro de ventas cabecera–detalle y la consulta de ventas, conectados a PharmaBackend.

Reto 02 (actividad autónoma): probar cálculos, validaciones, confirmaciones y mensajes, y construir la vista del reporte de ventas por categoría.

1.1 Resultado de aprendizaje y criterio que se trabaja

Elemento	Descripción
Resultado de aprendizaje de la unidad	Construye una SPA empresarial conectada al backend REST, con layout y navegación, CRUD de tablas independientes y dependientes, formularios cabecera–detalle, consultas y reportes; integra seguridad mediante JWT en el backend y gestión de sesión, guards e interceptores en el frontend.
Criterio 3 del producto	Implementa formularios transaccionales con cabecera–detalle, cálculos, validaciones, confirmación, consultas y reportes.

1.2 Punto de partida

Requisito	Cómo verificarlo
SPA PharmaSoft con Categorías y Productos fusionados en `develop`.	`git log --oneline develop` muestra tus commits de las sesiones 7 y 8.
PharmaBackend actualizado desde `master`.	Swagger muestra `POST /api/v1/ventas` y `GET /api/v1/ventas/buscar`.
Al menos tres clientes activos y cinco productos activos con stock.	Regístralos desde Swagger o desde tu módulo Clientes de la sesión 7.

2. Actividad práctica

Volvamos a la botica: el cajero necesita armar una boleta con varios productos, ver el total mientras la arma y no equivocarse al registrarla. Con el Reto 01 agregarás a PharmaSoft el módulo Ventas, con su formulario cabecera–detalle, su paso de confirmación y la consulta de ventas, paso a paso y con el acompañamiento del docente.

Campo	Contenido
Título	Módulo frontend: registro de ventas cabecera–detalle y consulta de ventas
Objetivo	Implementa en la SPA PharmaSoft un formulario transaccional de ventas con detalle dinámico que calcula subtotales y total, valida cada línea, solicita confirmación antes de registrar y se integra con una consulta de ventas filtrada por cliente, estado y fechas.
Duración	100 minutos de trabajo + 20 minutos de pruebas y retroalimentación

Paso 1. Actualizar los repositorios

```
# Backend
cd PharmaBackend
git checkout master
git pull

# Frontend
cd ../pharma-frontend
git checkout develop
git pull
git checkout -b feature/ventas-<apellido>
```

Paso 2. Revisar el contrato de Ventas

Prueba en Swagger los tres endpoints que usarás. Observa que el request de la venta lleva **solo identificadores y cantidades**: el precio, los subtotales y el total los calcula el backend con los precios vigentes.

Operación	Petición	Respuestas
Registrar venta	`POST /api/v1/ventas`	201 · 400 (sin cliente o sin detalles) · 404 (cliente o producto inexistente) · 409 (cliente o producto inactivo, stock insuficiente)
Consultar una venta	`GET /api/v1/ventas/{id}`	200 · 404
Buscar ventas	`GET /api/v1/ventas/buscar?clienteId=&estado=&desde=&hasta=&pagina=&tamanio=`	200 con página (vacía si no hay coincidencias) · 409 si `desde` es posterior a `hasta`

Ejemplo de cuerpo para registrar una venta:

```
{
  "clienteId": 1,
  "detalles": [
    { "productoId": 1, "cantidad": 3 },
    { "productoId": 3, "cantidad": 2 }
  ]
}
```

Lo que hace el backend al registrar

Verifica que el cliente exista y esté activo; para cada línea verifica el producto, descuenta el stock con un UPDATE condicional y calcula el subtotal; al final suma el total. Todo ocurre dentro de una transacción: si una línea falla (por ejemplo, stock insuficiente), **no se guarda nada** y el stock de las líneas anteriores se restaura.

Paso 3. Generar la estructura

```
ng g c features/ventas/pages/venta-form
ng g c features/ventas/pages/venta-list
ng g c features/ventas/pages/venta-detalle
ng g s features/ventas/services/venta-service
ng g interface features/ventas/models/venta --type=model
```

Si en la actividad autónoma de la sesión 7 ya construiste el módulo Clientes, reutiliza su modelo y su servicio. Si no, crea solo estos dos archivos mínimos, porque la venta necesita la lista de clientes:

Archivo: `src/app/features/clientes/models/cliente.model.ts`

```
export interface Cliente {
  id: number;
  dni: string;
  nombres: string;
  apellidos: string;
  email: string;
  telefono: string | null;
  direccion: string | null;
  estado: boolean;
}
```

Archivo: `src/app/features/clientes/services/cliente-service.ts`

```
import { inject, Service } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../../../../environments/environment';
import { PaginaResponse } from '../../../core/models/pagina-response';
import { Cliente } from '../models/cliente.model';

@Service()
export class ClienteService {
  private readonly http = inject(HttpClient);
  private readonly url = `${environment.apiUrl}/clientes`;

  listar(pagina = 0, tamanio = 20, ordenarPor = 'apellidos',
         direccion: 'asc' | 'desc' = 'asc'): Observable<PaginaResponse<Cliente>> {
    const params = new HttpParams()
      .set('pagina', pagina)
      .set('tamanio', tamanio)
      .set('ordenarPor', ordenarPor)
      .set('direccion', direccion);
    return this.http.get<PaginaResponse<Cliente>>(this.url, { params });
  }

  obtener(id: number): Observable<Cliente> {
    return this.http.get<Cliente>(`${this.url}/${id}`);
  }
}
```

Crea también una utilidad para redondear montos a dos decimales:

Archivo: `src/app/core/utils/numeros.ts`

```
/** Redondea a dos decimales (montos en soles). */
export function redondear(valor: number): number {
  return Math.round(valor * 100) / 100;
}
```

Y otra para los buscadores: compara sin distinguir mayúsculas ni tildes, de modo que «huaman» encuentre a «Huamán» y «para 500» encuentre «Paracetamol 500 mg».

Archivo: `src/app/core/utils/texto.ts`

```
/** Pasa a minúsculas y quita tildes: "Analgésicos" → "analgesicos". */
export function normalizar(texto: string): string {
  return texto.normalize('NFD').replace(/[̀-ͯ]/g, '').toLowerCase().trim();
}

/** true si todas las palabras buscadas aparecen en alguno de los campos. */
export function coincide(busqueda: string, ...campos: string[]): boolean {
  const palabras = normalizar(busqueda).split(/\s+/).filter(Boolean);
  const texto = normalizar(campos.join(' '));
  return palabras.every(p => texto.includes(p));
}
```

Paso 4. Modelos de la venta

Separa lo que **se recibe** (la venta con sus detalles calculados) de lo que **se envía** (cliente, productos y cantidades) y de los filtros de la consulta.

Archivo: `src/app/features/ventas/models/venta.model.ts`

```
export type EstadoVenta = 'REGISTRADA' | 'ANULADA';

/** Detalle tal como lo devuelve el backend (DetalleVentaResponseDTO). */
export interface DetalleVenta {
  productoId: number;
  productoNombre: string;
  cantidad: number;
  precio: number;
  subtotal: number;
}

/** Cabecera con sus detalles (VentaResponseDTO). */
export interface Venta {
  id: number;
  fecha: string;
  clienteId: number;
  clienteNombre: string;
  estado: EstadoVenta;
  total: number;
  detalles: DetalleVenta[];
}

/** Lo que se envía al registrar: solo ids y cantidades. */
export interface VentaRequest {
  clienteId: number;
  detalles: { productoId: number; cantidad: number }[];
}

/** Filtros opcionales de GET /ventas/buscar. */
export interface FiltroVentas {
  clienteId: number | null;
  estado: EstadoVenta | null;
  desde: string | null; // AAAA-MM-DD
  hasta: string | null;
}
```

Paso 5. Servicio HTTP

En `buscar()` solo se agregan a `HttpParams` los filtros que tienen valor. Así la URL refleja exactamente lo que el usuario eligió y, en la pestaña Red, es fácil comprobar qué filtros se aplicaron.

Archivo: `src/app/features/ventas/services/venta-service.ts`

```
import { inject, Service } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../../../../environments/environment';
import { PaginaResponse } from '../../../core/models/pagina-response';
import { FiltroVentas, Venta, VentaRequest } from '../models/venta.model';

@Service()
export class VentaService {
  private readonly http = inject(HttpClient);
  private readonly url = `${environment.apiUrl}/ventas`;

  registrar(dto: VentaRequest): Observable<Venta> {
    return this.http.post<Venta>(this.url, dto);
  }

  obtener(id: number): Observable<Venta> {
    return this.http.get<Venta>(`${this.url}/${id}`);
  }

  /** Solo se envían los filtros que tienen valor. */
  buscar(filtro: FiltroVentas, pagina: number, tamanio: number):
Observable<PaginaResponse<Venta>> {
    let params = new HttpParams()
      .set('pagina', pagina)
      .set('tamanio', tamanio)
      .set('ordenarPor', 'fecha')
      .set('direccion', 'desc');
    if (filtro.clienteId) params = params.set('clienteId', filtro.clienteId);
    if (filtro.estado) params = params.set('estado', filtro.estado);
    if (filtro.desde) params = params.set('desde', filtro.desde);
    if (filtro.hasta) params = params.set('hasta', filtro.hasta);
    return this.http.get<PaginaResponse<Venta>>(`${this.url}/buscar`, { params });
  }
}
```

Paso 6. Rutas y menú

Archivo: `src/app/features/ventas/ventas.routes.ts`

```
import { Routes } from '@angular/router';
import { VentaList } from './pages/venta-list/venta-list';
import { VentaForm } from './pages/venta-form/venta-form';
import { VentaDetalle } from './pages/venta-detalle/venta-detalle';

export const VENTAS_ROUTES: Routes = [
  { path: '', component: VentaList, title: 'Consulta de ventas' },
  { path: 'nueva', component: VentaForm, title: 'Nueva venta' },
  { path: ':id', component: VentaDetalle, title: 'Detalle de venta' },
];
```

En `app.routes.ts`, dentro de `children` de MainLayout:

```
{
  path: 'ventas',
  loadChildren: () =>
    import('./features/ventas/ventas.routes').then(m => m.VENTAS_ROUTES),
},
```

En `core/config/menu.ts`:

```
{ etiqueta: 'Ventas', ruta: '/ventas', icono: '🧾' },
```

Paso 7. Formulario cabecera–detalle

Este es el núcleo de la sesión. En una botica hay decenas de clientes y cientos de productos, así que el cajero no elige de una lista desplegable: **busca** al cliente por nombre o DNI y **busca** cada producto por nombre o categoría. Revisa en el código estas seis piezas:

- **Buscadores**: `buscarCliente` y `buscarProducto` son `FormControl` independientes, fuera del formulario de la venta, porque lo que se escribe para buscar no se envía al backend. `toSignal` los convierte en signals y `clientesEncontrados` y `productosEncontrados` son `computed` que filtran con `coincide()` y muestran hasta seis resultados.
- **FormArray** `detalles`: empieza vacío y cada elemento es un `FormGroup` con `productoId` y `cantidad`. `agregarProducto()` crea la línea desde el resultado elegido y `quitarLinea()` la elimina. `Validators.required` sobre el FormArray exige al menos una línea (con un arreglo vacío, `minLength(1)` no marcaría error).
- **Sin productos repetidos**: si el producto elegido ya está en el detalle, `agregarProducto()` suma 1 a su cantidad en vez de crear otra línea. Enter en el buscador agrega el primer resultado sin enviar el formulario.
- **Cálculos con signals**: `toSignal(form.valueChanges)` convierte el formulario en un signal; `lineas` y `total` son `computed` que se recalculan con cada cambio.
- **Validación por línea**: `excedeStock` compara la cantidad con el stock del producto.
- **Confirmación**: `revisar()` valida y muestra un resumen; solo `confirmar()` envía el POST.
- **Respuesta del servidor**: si el backend devuelve 409 (por ejemplo, stock insuficiente porque otra caja vendió antes), se cierra la confirmación y se muestra el mensaje.

Archivo: `src/app/features/ventas/pages/venta-form/venta-form.ts`

```
import { Component, computed, inject, OnInit, signal } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { CurrencyPipe } from '@angular/common';
import { HttpErrorResponse } from '@angular/common/http';
import { FormControl, FormGroup, NonNullableFormBuilder, ReactiveFormsModule, Validators } from
'@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { forkJoin } from 'rxjs';
import { mensajeError } from '../../../../core/utils/http-error';
import { redondear } from '../../../../core/utils/numeros';
import { coincide } from '../../../../core/utils/texto';
import { Cliente } from '../../../clientes/models/cliente.model';
import { ClienteService } from '../../../clientes/services/cliente-service';
import { Producto } from '../../../productos/models/producto.model';
import { ProductoService } from '../../../productos/services/producto-service';
import { VentaRequest } from '../../models/venta.model';
import { VentaService } from '../../services/venta-service';

/** Controles de una línea de detalle. */
interface LineaForm {
  productoId: FormControl<number | null>;
  cantidad: FormControl<number>;
}

const MAX_RESULTADOS = 6;

@Component({
  selector: 'app-venta-form',
  imports: [ReactiveFormsModule, RouterLink, CurrencyPipe],
  templateUrl: './venta-form.html',
  styleUrl: './venta-form.css',
```

```
})
export class VentaForm implements OnInit {
  private readonly fb = inject(NonNullableFormBuilder);
  private readonly ventaService = inject(VentaService);
  private readonly clienteService = inject(ClienteService);
  private readonly productoService = inject(ProductoService);
  private readonly router = inject(Router);

  protected readonly clientes = signal<Cliente[]>([]);
  protected readonly productos = signal<Producto[]>([]);
  protected readonly cargando = signal(true);
  protected readonly confirmando = signal(false);
  protected readonly guardando = signal(false);
  protected readonly error = signal<string | null>(null);
  /** Si está marcado, el comprobante se imprime apenas se registra la venta. */
  protected readonly imprimirAlRegistrar = signal(true);

  // ---------- Cabecera + detalle dinámico ----------
  // El detalle empieza vacío: las líneas se crean desde el buscador de productos.
  protected readonly form = this.fb.group({
    clienteId: this.fb.control<number | null>(null, Validators.required),
    detalles: this.fb.array<FormGroup<LineaForm>>([], Validators.required),
  });

  get detalles() {
    return this.form.controls.detalles;
  }

  private nuevaLinea(productoId: number): FormGroup<LineaForm> {
    return this.fb.group({
      productoId: this.fb.control<number | null>(productoId, Validators.required),
      cantidad: this.fb.control(1, [Validators.required, Validators.min(1),
Validators.pattern(/^\d+$/)]),
    });
  }

  quitarLinea(indice: number): void {
    this.detalles.removeAt(indice);
  }

  // ---------- Buscadores (controles fuera del formulario de la venta) ----------
  protected readonly buscarCliente = new FormControl('', { nonNullable: true });
  protected readonly buscarProducto = new FormControl('', { nonNullable: true });

  private readonly textoCliente = toSignal(this.buscarCliente.valueChanges, { initialValue: '' });
  private readonly textoProducto = toSignal(this.buscarProducto.valueChanges, { initialValue: ''
});

  /** Clientes que coinciden por nombres, apellidos o DNI. */
  protected readonly clientesEncontrados = computed(() => {
    const texto = this.textoCliente().trim();
    if (!texto) return [];
    return this.clientes()
      .filter(c => coincide(texto, c.nombres, c.apellidos, c.dni))
      .slice(0, MAX_RESULTADOS);
  });

  /** Productos que coinciden por nombre o categoría. */
  protected readonly productosEncontrados = computed(() => {
    const texto = this.textoProducto().trim();
    if (!texto) return [];
    return this.productos()
      .filter(p => coincide(texto, p.nombre, p.categoriaNombre))
      .slice(0, MAX_RESULTADOS);
  });

  protected readonly hayBusquedaCliente = computed(() => this.textoCliente().trim().length > 0);
  protected readonly hayBusquedaProducto = computed(() => this.textoProducto().trim().length > 0);

  elegirCliente(cliente: Cliente): void {
    this.form.controls.clienteId.setValue(cliente.id);
    this.buscarCliente.setValue('');
  }

  cambiarCliente(): void {
    this.form.controls.clienteId.setValue(null);
  }
```

```
  /** Si el producto ya está en el detalle, suma 1 a su cantidad; si no, crea la línea. */
  agregarProducto(producto: Producto): void {
    const linea = this.detalles.controls.find(l => l.controls.productoId.value === producto.id);
    if (linea) {
      linea.controls.cantidad.setValue(Number(linea.controls.cantidad.value) + 1);
    } else {
      this.detalles.push(this.nuevaLinea(producto.id));
    }
    this.buscarProducto.setValue('');
  }

  /** Enter en el buscador agrega el primer resultado sin enviar el formulario. */
  agregarPrimero(evento: Event): void {
    evento.preventDefault();
    const primero = this.productosEncontrados()[0];
    if (primero) this.agregarProducto(primero);
  }

  /** Cantidad que ya tiene un producto en el detalle (0 si no está). */
  cantidadEnDetalle(productoId: number): number {
    const linea = this.valor().detalles?.find(d => d.productoId === productoId);
    return linea ? Number(linea.cantidad) || 0 : 0;
  }

  // ---------- Cálculos a partir del valor del formulario ----------
  private readonly valor = toSignal(this.form.valueChanges, { initialValue: this.form.value });

  private readonly productoPorId = computed(() =>
    new Map(this.productos().map(p => [p.id, p])));

  protected readonly lineas = computed(() =>
    (this.valor().detalles ?? []).map(d => {
      const producto = d.productoId ? this.productoPorId().get(d.productoId) : undefined;
      const cantidad = Number(d.cantidad) || 0;
      const precio = producto?.precio ?? 0;
      return {
        producto,
        precio,
        subtotal: redondear(precio * cantidad),
        excedeStock: !!producto && cantidad > producto.stock,
      };
    }));

  protected readonly total = computed(() =>
    redondear(this.lineas().reduce((suma, l) => suma + l.subtotal, 0)));

  protected readonly hayExcesoDeStock = computed(() => this.lineas().some(l => l.excedeStock));

  protected readonly clienteElegido = computed(() =>
    this.clientes().find(c => c.id === this.valor().clienteId));

  // ---------- Carga inicial ----------
  ngOnInit(): void {
    forkJoin({
      clientes: this.clienteService.listar(0, 100),
      productos: this.productoService.listar(0, 100, 'nombre', 'asc'),
    }).subscribe({
      next: ({ clientes, productos }) => {
        this.clientes.set(clientes.contenido.filter(c => c.estado));
        this.productos.set(productos.contenido.filter(p => p.estado && p.stock > 0));
        this.cargando.set(false);
      },
      error: (err: HttpErrorResponse) => {
        this.error.set(mensajeError(err));
        this.cargando.set(false);
      },
    });
  }

  // ---------- Revisión, confirmación y registro ----------
  revisar(): void {
    if (this.form.invalid || this.hayExcesoDeStock()) {
      this.form.markAllAsTouched();
      return;
    }
    this.error.set(null);
```

```
    this.confirmando.set(true);
  }

  confirmar(): void {
    const v = this.form.getRawValue();
    const dto: VentaRequest = {
      clienteId: Number(v.clienteId),
      detalles: v.detalles.map(d => ({ productoId: Number(d.productoId), cantidad:
Number(d.cantidad) })),
    };
    this.guardando.set(true);
    this.ventaService.registrar(dto).subscribe({
      next: venta => this.router.navigate(['/ventas', venta.id], {
        queryParams: this.imprimirAlRegistrar() ? { nueva: 1, imprimir: 1 } : { nueva: 1 },
      }),
      error: (err: HttpErrorResponse) => {
        this.guardando.set(false);
        this.confirmando.set(false);
        this.error.set(mensajeError(err));
      },
    });
  }
}
```

En la plantilla, los buscadores se enlazan con `[formControl]` y sus resultados se recorren con `@for … @empty`, que muestra un aviso cuando nada coincide. Una vez elegido el cliente, el buscador se reemplaza por una ficha con el botón «Cambiar». En la tabla, `formArrayName="detalles"` y `[formGroupName]="i"` conectan cada fila con su línea del FormArray; el nombre, el precio y el subtotal se leen de `lineas()[i]`.

Archivo: `src/app/features/ventas/pages/venta-form/venta-form.html`

```
<section class="page">
  <h2>Nueva venta</h2>

  @if (error()) {
    <div class="alerta">{{ error() }}</div>
  }

  @if (cargando()) {
    <p>Cargando clientes y productos…</p>
  } @else {
    <form [formGroup]="form" (ngSubmit)="revisar()" class="card">
      <!-- Cabecera: buscador de cliente -->
      <h3 class="subtitulo">1. Cliente</h3>
      @if (clienteElegido(); as c) {
        <div class="elegido">
          <span><strong>{{ c.apellidos }}, {{ c.nombres }}</strong> · DNI {{ c.dni }}</span>
          <button type="button" class="btn" (click)="cambiarCliente()">Cambiar</button>
        </div>
      } @else {
        <div class="buscador">
          <input class="input" type="search" [formControl]="buscarCliente"
                 placeholder="Buscar cliente por nombre, apellido o DNI…" aria-label="Buscar
cliente" />
          @if (hayBusquedaCliente()) {
            <ul class="resultados">
              @for (c of clientesEncontrados(); track c.id) {
                <li>
                  <button type="button" (click)="elegirCliente(c)">
                    <strong>{{ c.apellidos }}, {{ c.nombres }}</strong>
                    <span class="dato">DNI {{ c.dni }}</span>
                  </button>
                </li>
              } @empty {
                <li class="vacio">Ningún cliente activo coincide con la búsqueda.</li>
              }
            </ul>
          }
        </div>
        @if (form.controls.clienteId.touched && form.controls.clienteId.invalid) {
          <small class="error">Busque y seleccione el cliente.</small>
        }
      }
```

```
      <!-- Detalle: buscador de productos -->
      <h3 class="subtitulo">2. Productos</h3>
      <div class="buscador">
        <input class="input" type="search" [formControl]="buscarProducto"
               (keydown.enter)="agregarPrimero($event)"
               placeholder="Buscar producto por nombre o categoría… (Enter agrega el primero)"
               aria-label="Buscar producto" />
        @if (hayBusquedaProducto()) {
          <ul class="resultados">
            @for (p of productosEncontrados(); track p.id) {
              <li>
                <button type="button" (click)="agregarProducto(p)">
                  <strong>{{ p.nombre }}</strong>
                  <span class="dato">{{ p.categoriaNombre }}</span>
                  <span class="dato">{{ p.precio | currency: 'PEN' : 'S/ ' }}</span>
                  <span class="dato">Stock {{ p.stock }}</span>
                  @if (cantidadEnDetalle(p.id); as n) {
                    <span class="ya">En la venta: {{ n }}</span>
                  }
                  <span class="agregar">+ Agregar</span>
                </button>
              </li>
            } @empty {
              <li class="vacio">Ningún producto activo con stock coincide con la búsqueda.</li>
            }
          </ul>
        }
      </div>

      <table class="tabla detalle" formArrayName="detalles">
        <thead>
          <tr><th>Producto</th><th>Precio</th><th>Cantidad</th><th>Subtotal</th><th></th></tr>
        </thead>
        <tbody>
          @for (linea of detalles.controls; track linea; let i = $index) {
            <tr [formGroupName]="i">
              <td>
                {{ lineas()[i]?.producto?.nombre }}
                <small class="dato">{{ lineas()[i]?.producto?.categoriaNombre }}</small>
              </td>
              <td>{{ lineas()[i]?.precio | currency: 'PEN' : 'S/ ' }}</td>
              <td>
                <input class="input cantidad" type="number" min="1" step="1"
formControlName="cantidad" />
                @if (lineas()[i]?.excedeStock) {
                  <small class="error">Supera el stock ({{ lineas()[i].producto?.stock }})</small>
                }
              </td>
              <td class="monto">{{ lineas()[i]?.subtotal | currency: 'PEN' : 'S/ ' }}</td>
              <td>
                <button type="button" class="btn btn-peligro" (click)="quitarLinea(i)"
                        aria-label="Quitar producto">✕</button>
              </td>
            </tr>
          } @empty {
            <tr><td colspan="5" class="vacio">Aún no hay productos. Use el buscador para
agregarlos.</td></tr>
          }
        </tbody>
        <tfoot>
          <tr>
            <td colspan="3" class="monto">Total estimado</td>
            <td class="monto total">{{ total() | currency: 'PEN' : 'S/ ' }}</td>
            <td></td>
          </tr>
        </tfoot>
      </table>

      @if (form.controls.detalles.touched && form.controls.detalles.invalid) {
        <small class="error">Agregue al menos un producto y una cantidad entera mayor que cero en
cada línea.</small>
      }

      <div class="acciones">
        <a routerLink="/ventas" class="btn">Cancelar</a>
        <button type="submit" class="btn btn-primary">Revisar venta</button>
      </div>
    </form>
```

```
    <!-- Confirmación de la operación -->
    @if (confirmando()) {
      <div class="confirmacion card" role="dialog" aria-label="Confirmar venta">
        <h3>¿Confirma el registro de la venta?</h3>
        <p>Cliente: <strong>{{ clienteElegido()?.apellidos }}, {{ clienteElegido()?.nombres
}}</strong></p>
        <p>Productos: <strong>{{ detalles.length }}</strong> · Total estimado: <strong>{{ total() |
currency: 'PEN' : 'S/ ' }}</strong></p>
        <p class="nota">El total definitivo lo calcula el servidor con los precios vigentes.</p>
        <label class="check">
          <input type="checkbox" [checked]="imprimirAlRegistrar()"
                 (change)="imprimirAlRegistrar.set($any($event.target).checked)" />
          Imprimir el comprobante al registrar
        </label>
        <div class="acciones">
          <button type="button" class="btn" [disabled]="guardando()"
(click)="confirmando.set(false)">Volver</button>
          <button type="button" class="btn btn-primary" [disabled]="guardando()"
(click)="confirmar()">
            {{ guardando() ? 'Registrando…' : 'Confirmar venta' }}
          </button>
        </div>
      </div>
    }
  }
</section>
```

Archivo: `src/app/features/ventas/pages/venta-form/venta-form.css`

```
.subtitulo { margin: 8px 0 8px; font-size: 16px; color: #0f2e5c; }
.subtitulo:not(:first-child) { margin-top: 20px; }

/* Buscadores */
.buscador { position: relative; max-width: 640px; }
.buscador .input { width: 100%; }
.resultados {
  list-style: none; margin: 4px 0 0; padding: 4px; border: 1px solid #c9d3e3;
  border-radius: 8px; background: #fff; box-shadow: 0 6px 16px rgba(15, 46, 92, .12);
}
.resultados button {
  width: 100%; display: flex; flex-wrap: wrap; align-items: baseline; gap: 4px 12px;
  padding: 8px 10px; border: 0; border-radius: 6px; background: none; text-align: left;
  font: inherit; cursor: pointer;
}
.resultados button:hover, .resultados button:focus-visible { background: #e9f0f9; outline: none; }
.dato { color: #4b5563; font-size: 14px; }
.ya { color: #1f5aa6; font-size: 13px; font-weight: 600; }
.agregar { margin-left: auto; color: #0f2e5c; font-weight: 700; }
.vacio { padding: 8px 10px; color: #6b7280; }
.detalle .vacio { text-align: center; padding: 16px; }

.elegido {
  display: flex; align-items: center; justify-content: space-between; gap: 12px; max-width: 640px;
  padding: 10px 12px; border: 1px solid #b7c8e0; border-radius: 8px; background: #e9f0f9;
}

.detalle { margin-top: 16px; }
.detalle td { vertical-align: top; }
.detalle td small.dato { display: block; }
.cantidad { width: 90px; }
.monto { text-align: right; white-space: nowrap; }
.total { font-size: 18px; font-weight: 700; color: #0f2e5c; }
.error { display: block; color: #b42318; margin-top: 4px; }
.acciones { display: flex; justify-content: flex-end; gap: 8px; margin-top: 16px; }
.confirmacion { margin-top: 16px; border: 2px solid #f2a900; }
.confirmacion h3 { margin-top: 0; color: #0f2e5c; }
.nota { color: #6b7280; font-size: 14px; }
.check { display: flex; align-items: center; gap: 8px; margin-top: 8px; font-weight: 600; }
```

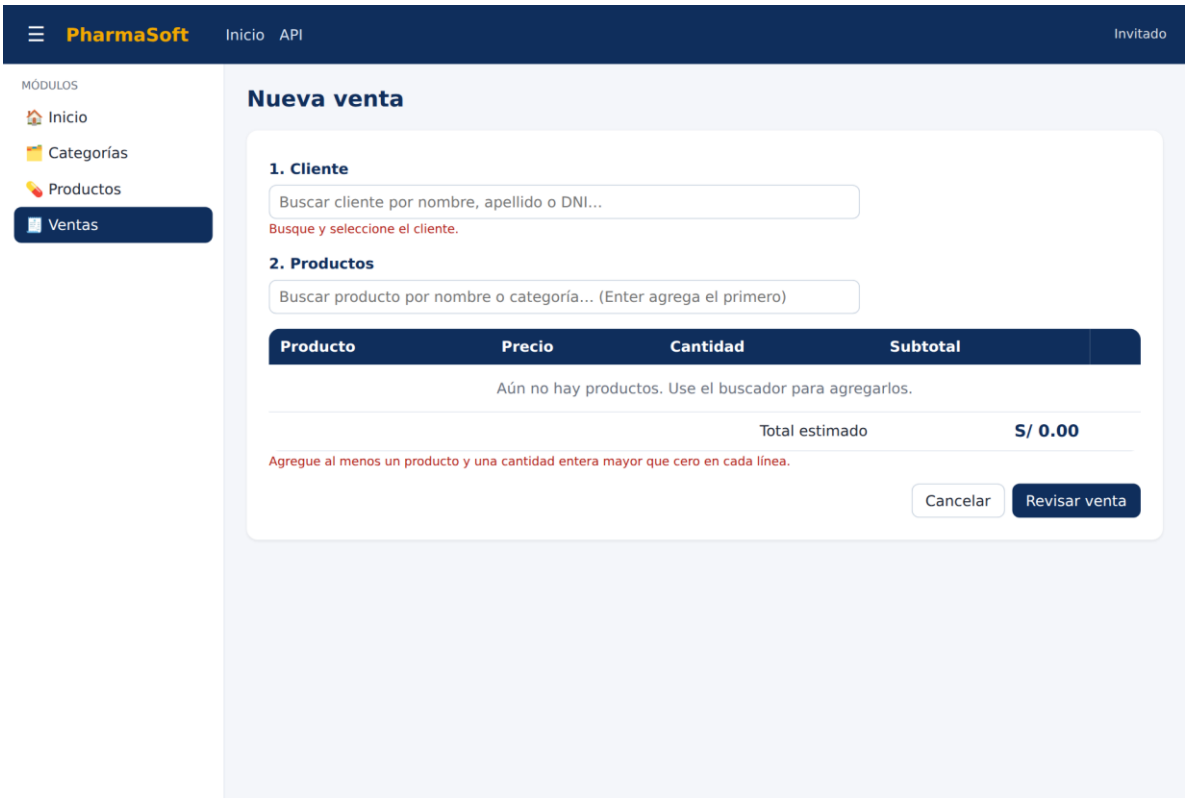


Figura 1. Validaciones de cabecera y detalle antes de revisar la venta. Nota. Elaboración propia.

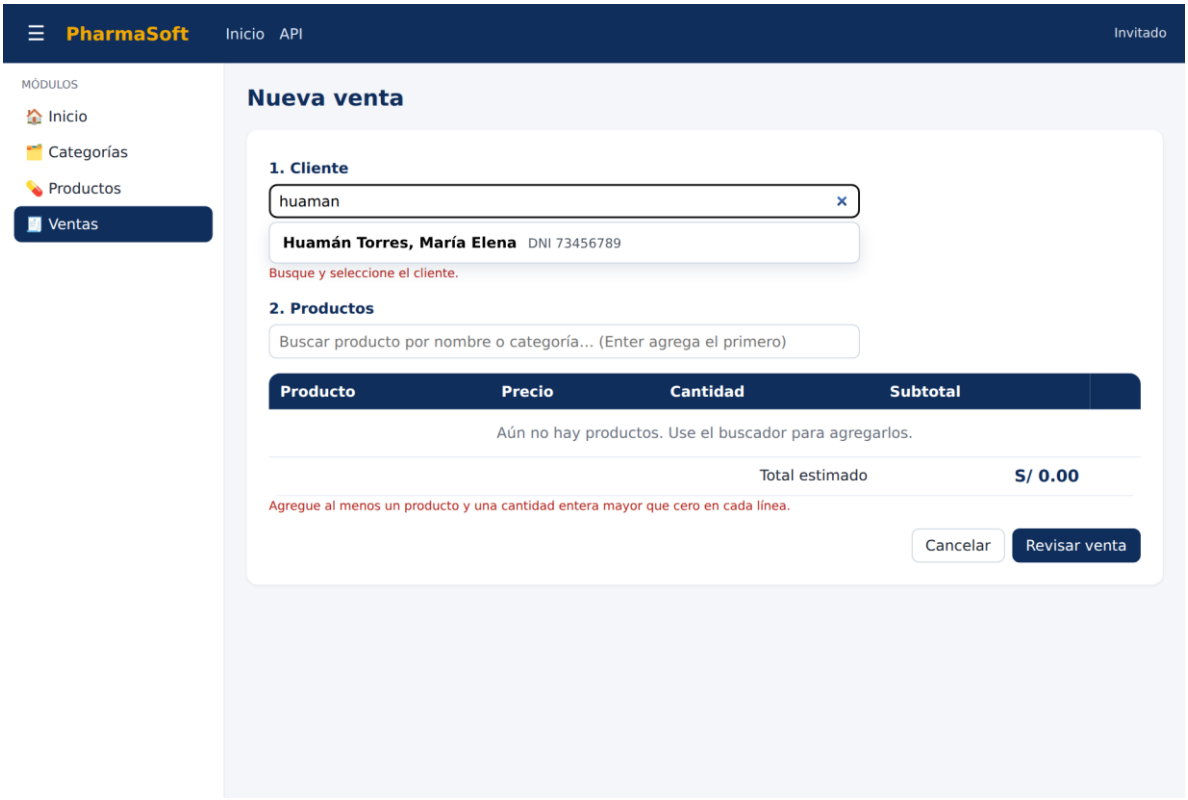


Figura 2. Búsqueda del cliente sin distinguir tildes («huaman» encuentra a Huamán). Nota. Elaboración propia.

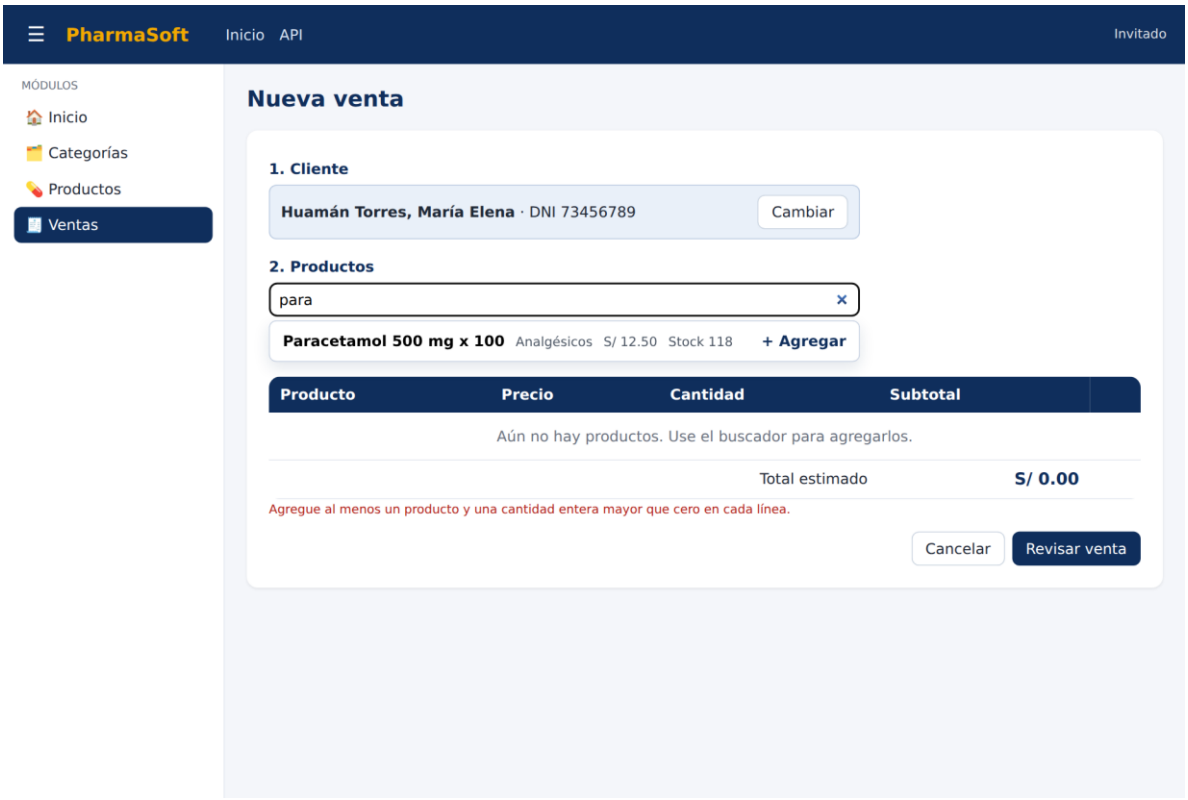


Figura 3. Búsqueda de productos con precio, categoría y stock. Nota. Elaboración propia.

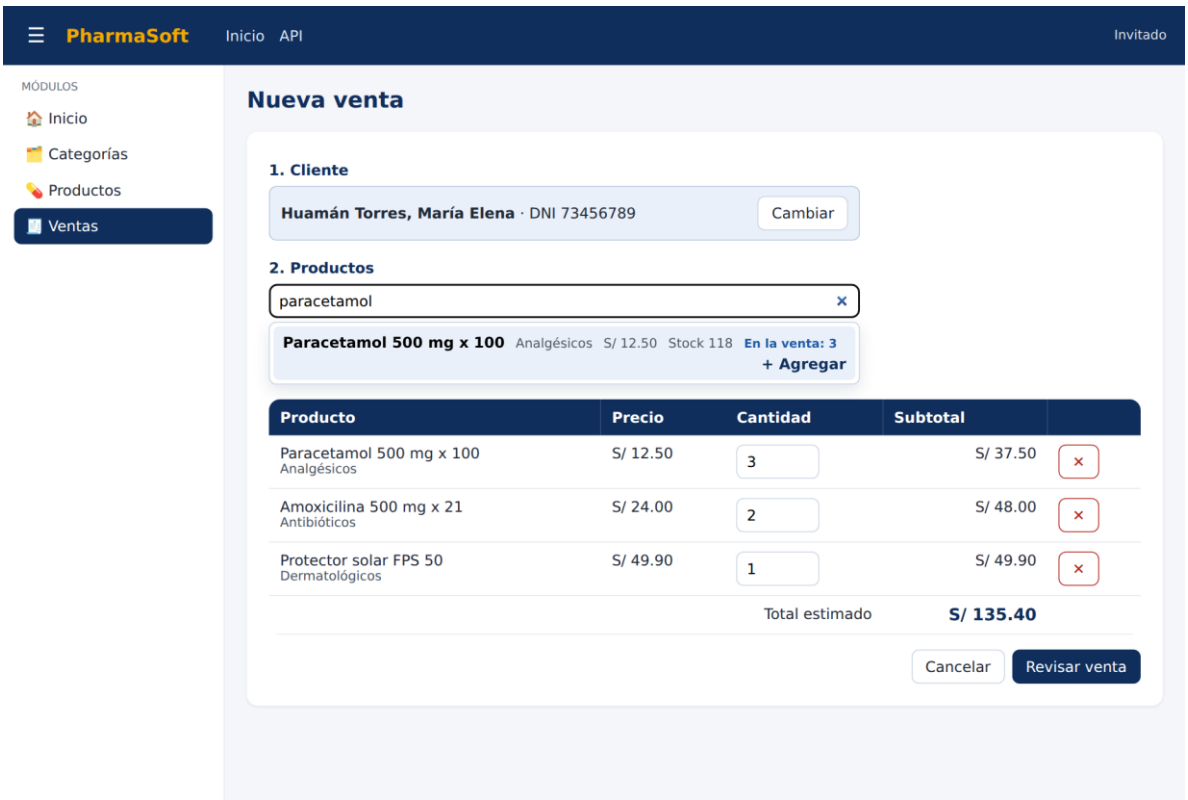


Figura 4. El buscador avisa que el producto ya está en la venta; al agregarlo sube su cantidad. Nota. Elaboración propia.

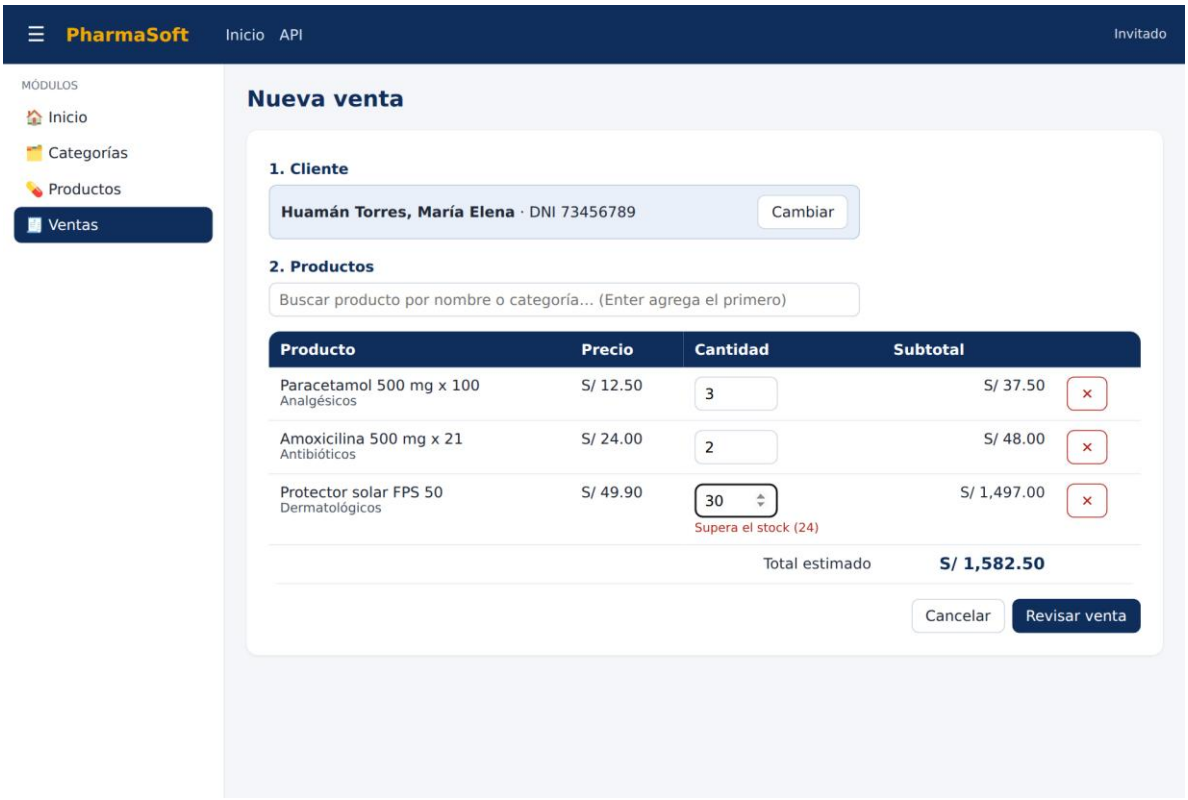


Figura 5. La cantidad de una línea supera el stock disponible. Nota. Elaboración propia.

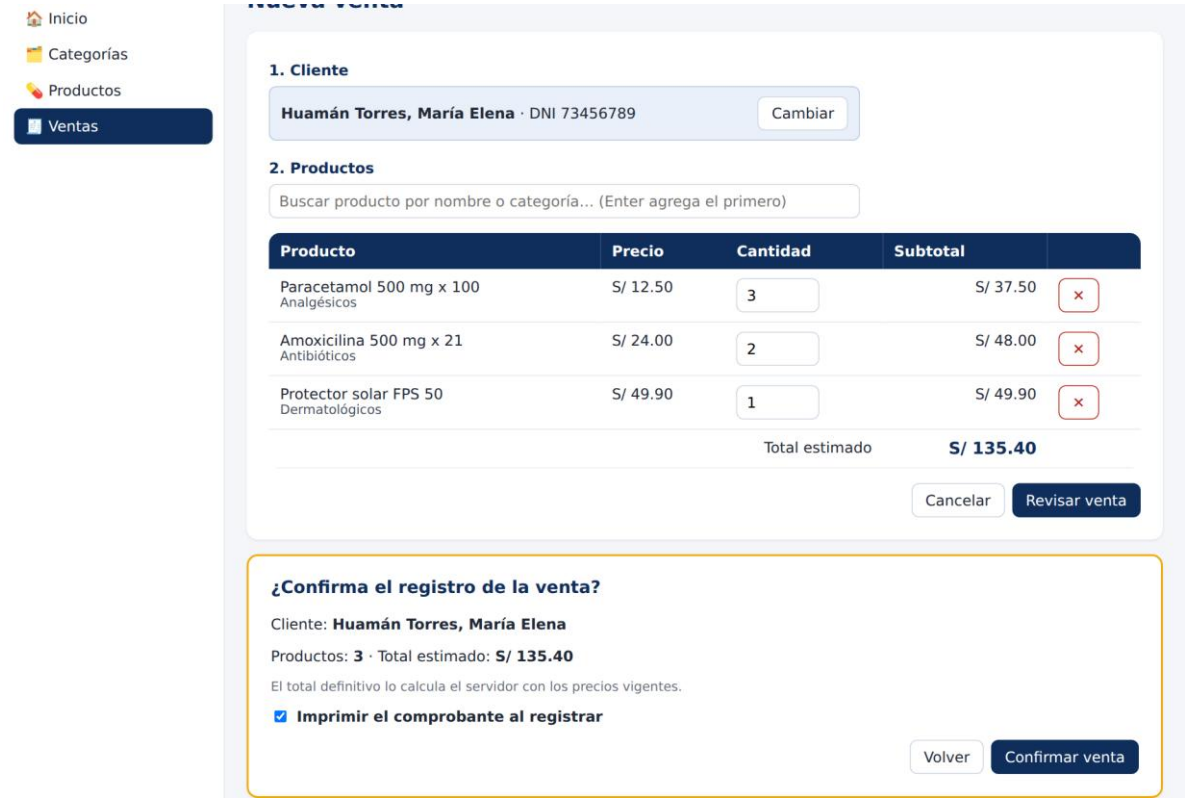


Figura 6. Resumen y confirmación de la venta. Nota. Elaboración propia.

Paso 8. Comprobante de la venta e impresión

Después de registrar, la SPA navega a `/ventas/{id}?nueva=1`. El componente recibe `id`, `nueva` e `imprimir` como inputs y muestra la venta tal como la guardó el servidor, con su total definitivo. Como la venta solo trae el nombre del cliente, el componente pide además el cliente para mostrar su DNI; si esa petición falla, el comprobante sale igual, sin DNI.

La botica necesita entregar el comprobante en papel. Para imprimir no hace falta ninguna librería: `window.print()` abre el diálogo del navegador (desde allí también se puede guardar como PDF) y las reglas `@media print` deciden qué sale en el papel:

- **Fuera del papel**: el menú y la cabecera de la SPA, los botones y los filtros (clase `.no-imprimir`).
- **Solo en el papel**: el pie «Gracias por su compra» con la fecha de impresión (clase `.solo-impresion`).
- **Impresión automática**: en la confirmación, la casilla «Imprimir el comprobante al registrar» agrega `imprimir=1` a la URL; el comprobante llama a `window.print()` cuando ya tiene los datos. En la consulta, el botón 🖶 de cada fila abre el comprobante con ese mismo parámetro.
- **Venta anulada**: el comprobante muestra el sello «ANULADA» en pantalla y en el papel.

Primero agrega `obtener()` al servicio de clientes (`GET /api/v1/clientes/{id}` ya existe en PharmaBackend):

```
obtener(id: number): Observable<Cliente> {
  return this.http.get<Cliente>(`${this.url}/${id}`);
}
```

Reglas de impresión comunes, al final de `src/styles.css`:

```
/* ---------- Impresión ---------- */
.solo-impresion { display: none; }
@media print {
  @page { size: A4; margin: 12mm; }
  * { -webkit-print-color-adjust: exact; print-color-adjust: exact; }
  body { background: #fff; }
  /* Fuera del papel: botones, filtros y todo lo marcado .no-imprimir (el layout oculta su
menú en main-layout.css) */
  .no-imprimir { display: none !important; }
  .solo-impresion { display: block; }
  .card { box-shadow: none; border-radius: 0; padding: 0; }
  .tabla { border-radius: 0; }
  .tabla th { background: #fff; color: #000; border-bottom: 2px solid #0f2e5c; }
  .tabla tr { break-inside: avoid; }
}
```

El menú se oculta en `main-layout.css` y no en `styles.css`: los estilos de un componente se encapsulan y tienen más prioridad que los globales sobre sus propios elementos.

```
/* Al imprimir solo queda el contenido de la página: sin cabecera ni menú. */
@media print {
  .shell { display: block; min-height: 0; }
  .area-header, .area-sidebar { display: none; }
  .area-main { padding: 0; background: #fff; }
}
```

Archivo: `src/app/features/ventas/pages/venta-detalle/venta-detalle.ts`

```
import { Component, computed, inject, input, OnInit, signal } from '@angular/core';
import { CurrencyPipe, DatePipe } from '@angular/common';
import { HttpErrorResponse } from '@angular/common/http';
import { RouterLink } from '@angular/router';
import { catchError, map, of, switchMap } from 'rxjs';
import { mensajeError } from '../../../../core/utils/http-error';
import { Cliente } from '../../../clientes/models/cliente.model';
```

```
import { ClienteService } from '../../../clientes/services/cliente-service';
import { Venta } from '../../models/venta.model';
import { VentaService } from '../../services/venta-service';

@Component({
  selector: 'app-venta-detalle',
  imports: [RouterLink, CurrencyPipe, DatePipe],
  templateUrl: './venta-detalle.html',
  styleUrl: './venta-detalle.css',
})
export class VentaDetalle implements OnInit {
  private readonly ventaService = inject(VentaService);
  private readonly clienteService = inject(ClienteService);

  readonly id = input.required<string>();
  /** Llega como ?nueva=1 después de registrar. */
  readonly nueva = input<string>();
  /** Llega como ?imprimir=1 cuando se pide imprimir al abrir. */
  readonly imprimir = input<string>();

  protected readonly venta = signal<Venta | null>(null);
  protected readonly cliente = signal<Cliente | null>(null);
  protected readonly error = signal<string | null>(null);
  protected readonly fechaImpresion = new Date();

  /** Número con ceros a la izquierda: 4 → "000004". */
  protected readonly numero = computed(() => String(this.venta()?.id ?? '').padStart(6,
'0'));
  protected readonly unidades = computed(() =>
    (this.venta()?.detalles ?? []).reduce((suma, d) => suma + d.cantidad, 0));

  ngOnInit(): void {
    this.ventaService.obtener(Number(this.id())).pipe(
      // El DNI no viene en la venta: se pide el cliente. Si falla, el comprobante sale sin
DNI.
      switchMap(venta => this.clienteService.obtener(venta.clienteId).pipe(
        catchError(() => of(null)),
        map(cliente => ({ venta, cliente })),
      )),
    ).subscribe({
      next: ({ venta, cliente }) => {
        this.venta.set(venta);
        this.cliente.set(cliente);
        // Se espera a que Angular pinte el comprobante antes de abrir el diálogo de
impresión.
        if (this.imprimir()) setTimeout(() => window.print(), 300);
      },
      error: (err: HttpErrorResponse) => this.error.set(mensajeError(err)),
    });
  }

  protected imprimirComprobante(): void {
    window.print();
  }
}
```

Archivo: `src/app/features/ventas/pages/venta-detalle/venta-detalle.html`

```
<section class="page">
  @if (nueva()) {
    <div class="exito no-imprimir">Venta registrada correctamente.</div>
  }
  @if (error()) {
    <div class="alerta">{{ error() }}</div>
  }

  @if (venta(); as v) {
    <article class="card comprobante" [class.anulada]="v.estado === 'ANULADA'">
      <!-- Datos del negocio: se ven en pantalla y en el papel -->
      <header class="negocio">
        <div>
```

```
          <strong class="marca">PharmaSoft</strong>
          <span>Botica · Sistema de ventas</span>
        </div>
        <div class="numero">
          <span>Comprobante de venta</span>
          <strong>N.° {{ numero() }}</strong>
        </div>
      </header>

      <dl class="datos">
        <div><dt>Fecha</dt><dd>{{ v.fecha | date: 'dd/MM/yyyy HH:mm' }}</dd></div>
        <div><dt>Cliente</dt><dd>{{ v.clienteNombre }}</dd></div>
        @if (cliente(); as c) {
          <div><dt>DNI</dt><dd>{{ c.dni }}</dd></div>
        }
        <div>
          <dt>Estado</dt>
          <dd><span class="badge" [class.inactivo]="v.estado === 'ANULADA'">{{ v.estado
}}</span></dd>
        </div>
      </dl>

      <table class="tabla">
        <thead>
          <tr><th>Producto</th><th class="monto">Precio</th><th class="monto">Cant.</th><th
class="monto">Subtotal</th></tr>
        </thead>
        <tbody>
          @for (d of v.detalles; track d.productoId) {
            <tr>
              <td>{{ d.productoNombre }}</td>
              <td class="monto">{{ d.precio | currency: 'PEN' : 'S/ ' }}</td>
              <td class="monto">{{ d.cantidad }}</td>
              <td class="monto">{{ d.subtotal | currency: 'PEN' : 'S/ ' }}</td>
            </tr>
          }
        </tbody>
        <tfoot>
          <tr>
            <td colspan="2">{{ v.detalles.length }} {{ v.detalles.length === 1 ? 'producto' :
'productos' }} · {{ unidades() }} {{ unidades() === 1 ? 'unidad' : 'unidades' }}</td>
            <td class="monto"><strong>Total</strong></td>
            <td class="monto total">{{ v.total | currency: 'PEN' : 'S/ ' }}</td>
          </tr>
        </tfoot>
      </table>

      @if (v.estado === 'ANULADA') {
        <p class="sello" aria-hidden="true">ANULADA</p>
      }

      <!-- Solo en el papel -->
      <footer class="pie solo-impresion">
        <p>Gracias por su compra.</p>
        <p>Documento interno sin valor tributario · Impreso el {{ fechaImpresion | date:
'dd/MM/yyyy HH:mm' }}</p>
      </footer>

      <div class="acciones no-imprimir">
        <a routerLink="/ventas" class="btn">Volver a la consulta</a>
        <a routerLink="/ventas/nueva" class="btn">Nueva venta</a>
        <button type="button" class="btn btn-primary" (click)="imprimirComprobante()">
Imprimir</button>
      </div>
    </article>
  }
</section>
```

Archivo: `src/app/features/ventas/pages/venta-detalle/venta-detalle.css`

```
.comprobante { position: relative; max-width: 760px; }
.negocio {
  display: flex; justify-content: space-between; align-items: flex-start; gap: 16px;
  padding-bottom: 12px; margin-bottom: 12px; border-bottom: 3px solid #f2a900;
}
.negocio div { display: flex; flex-direction: column; gap: 2px; }
.marca { font-size: 22px; color: #0f2e5c; }
.negocio span { color: #4b5563; font-size: 14px; }
.numero { text-align: right; align-items: flex-end; }
.numero strong { font-size: 20px; color: #0f2e5c; }

.datos { display: grid; grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); gap: 8px 16px;
margin: 0 0 16px; }
.datos dt { font-size: 12px; text-transform: uppercase; letter-spacing: .04em; color: #6b7280; }
.datos dd { margin: 2px 0 0; font-weight: 600; }

.monto { text-align: right; white-space: nowrap; }
.total { font-size: 18px; font-weight: 700; color: #0f2e5c; }
.exito { padding: 12px 16px; margin-bottom: 16px; border-radius: 8px; background: #dcfce7; color:
#166534; }
.acciones { display: flex; justify-content: flex-end; gap: 8px; margin-top: 16px; }

.sello {
  position: absolute; top: 40%; left: 50%; margin: 0; transform: translate(-50%, -50%) rotate(-
18deg);
  font-size: 64px; font-weight: 800; letter-spacing: .1em; color: rgb(180 35 24 / .18); pointer-
events: none;
}
.pie { margin-top: 24px; text-align: center; font-size: 12px; color: #4b5563; }
.pie p { margin: 2px 0; }

@media print {
  .comprobante { max-width: none; }
  .datos dd .badge { background: none; padding: 0; color: #000; }
}
```

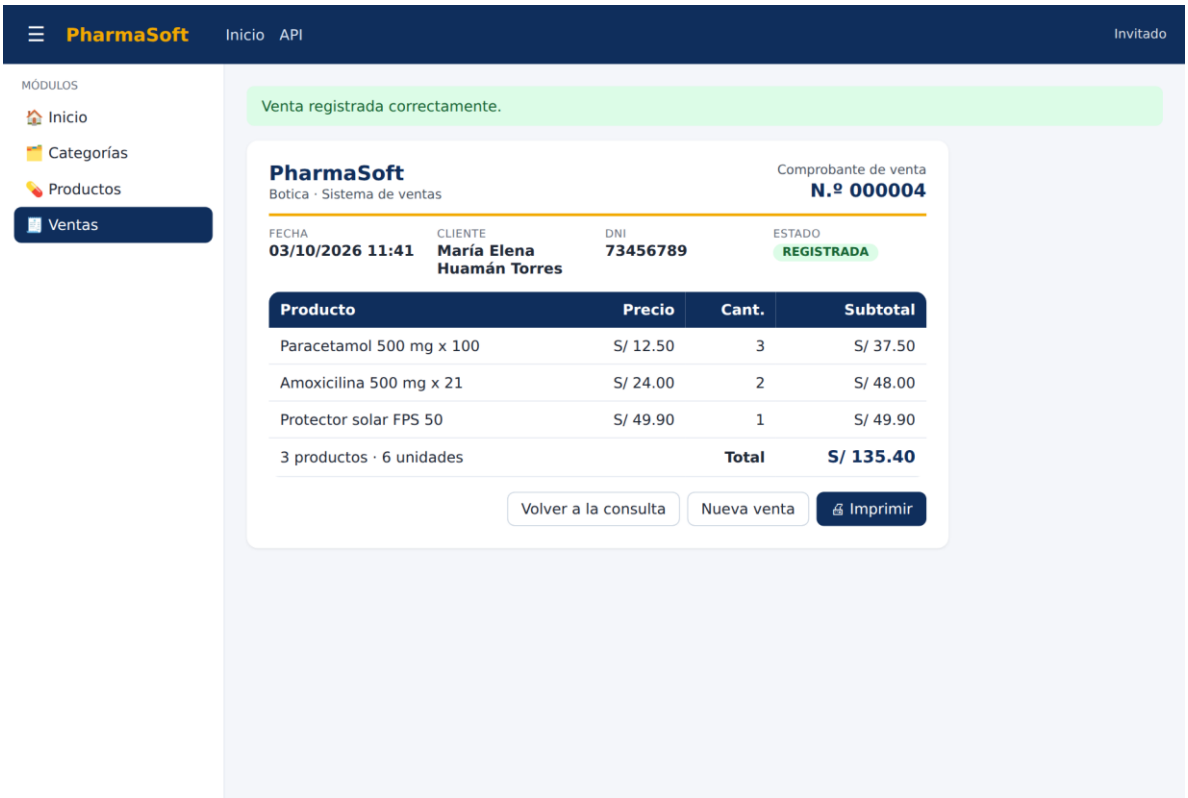
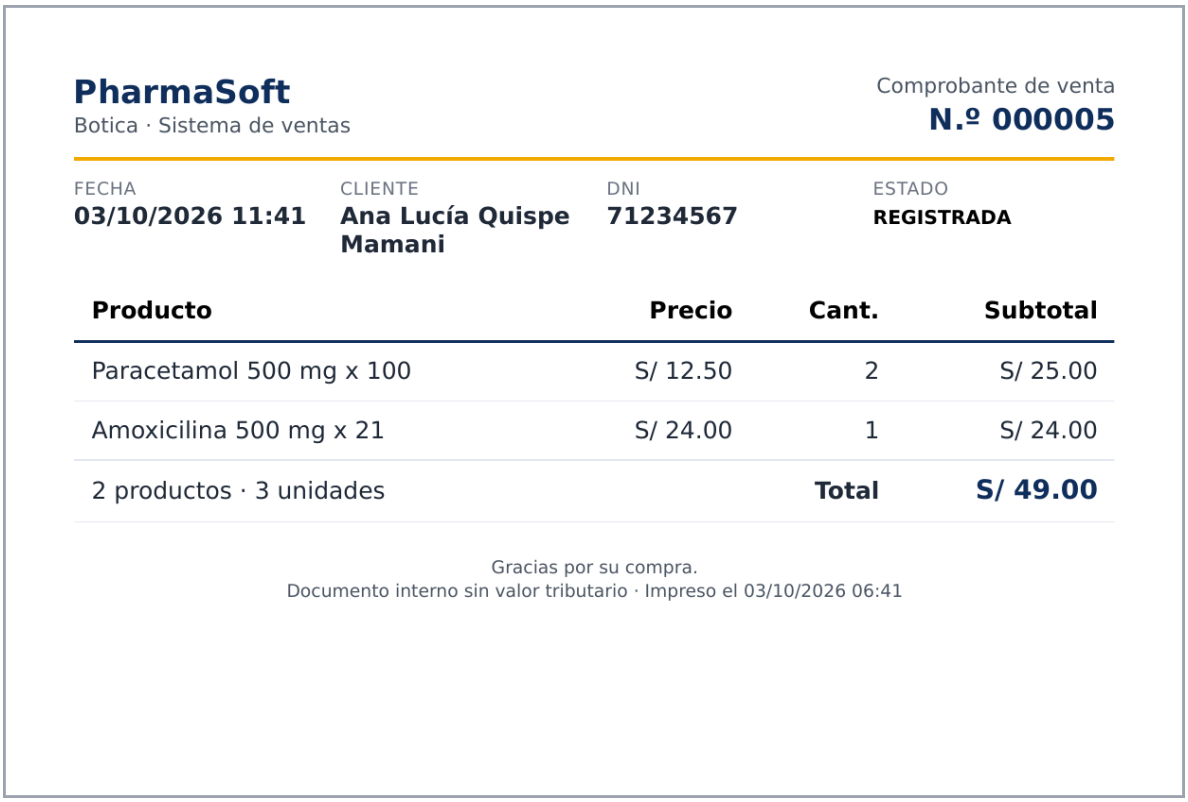


Figura 7. Comprobante de la venta registrada, en pantalla. Nota. Elaboración propia.



PharmaSoft
Botica · Sistema de ventas

Comprobante de venta
N.º 000005

FECHA	CLIENTE	DNI	ESTADO
03/10/2026 11:41	**Ana Lucía Quispe Mamani**	**71234567**	**REGISTRADA**

Producto	Precio	Cant.	Subtotal
Paracetamol 500 mg x 100	S/ 12.50	2	S/ 25.00
Amoxicilina 500 mg x 21	S/ 24.00	1	S/ 24.00
2 productos · 3 unidades		**Total**	**S/ 49.00**

Gracias por su compra.
Documento interno sin valor tributario · Impreso el 03/10/2026 06:41

Figura 8. Vista previa de impresión de un comprobante: sin menú ni botones, con DNI y pie de impresión. Nota. Elaboración propia.



PharmaSoft
Botica · Sistema de ventas

Comprobante de venta
N.º 000002

FECHA	CLIENTE	DNI	ESTADO
02/10/2026 11:40	**Jorge Luis Condori Apaza**	**72345678**	**ANULADA**

ANULADA

Producto	Precio	Cant.	Subtotal
Amoxicilina 500 mg x 21	S/ 24.00	1	S/ 24.00
1 producto · 1 unidad		**Total**	**S/ 24.00**

Gracias por su compra.
Documento interno sin valor tributario · Impreso el 03/10/2026 06:41

Figura 9. Comprobante impreso de una venta anulada. Nota. Elaboración propia.

Paso 9. Consulta de ventas

La consulta usa un formulario reactivo para los filtros. Antes de buscar, valida en la SPA que la fecha «desde» no sea posterior a «hasta»; el backend aplica la misma regla y respondería 409. El botón «Imprimir consulta»

imprime el listado sin filtros ni botones, con un encabezado que dice qué filtros se aplicaron y el total registrado de la página (sin contar las anuladas).

Archivo: `src/app/features/ventas/pages/venta-list/venta-list.ts`

```
import { Component, computed, inject, OnInit, signal } from '@angular/core';
import { CurrencyPipe, DatePipe } from '@angular/common';
import { HttpErrorResponse } from '@angular/common/http';
import { NonNullableFormBuilder, ReactiveFormsModule } from '@angular/forms';
import { RouterLink } from '@angular/router';
import { PaginaResponse } from '../../../../core/models/pagina-response';
import { mensajeError } from '../../../../core/utils/http-error';
import { Cliente } from '../../../clientes/models/cliente.model';
import { ClienteService } from '../../../clientes/services/cliente-service';
import { EstadoVenta, FiltroVentas, Venta } from '../../models/venta.model';
import { VentaService } from '../../services/venta-service';

@Component({
  selector: 'app-venta-list',
  imports: [ReactiveFormsModule, RouterLink, CurrencyPipe, DatePipe],
  templateUrl: './venta-list.html',
  styleUrl: './venta-list.css',
})
export class VentaList implements OnInit {
  private readonly fb = inject(NonNullableFormBuilder);
  private readonly ventaService = inject(VentaService);
  private readonly clienteService = inject(ClienteService);

  protected readonly clientes = signal<Cliente[]>([]);
  protected readonly resultado = signal<PaginaResponse<Venta> | null>(null);
  protected readonly pagina = signal(0);
  protected readonly cargando = signal(false);
  protected readonly error = signal<string | null>(null);

  protected readonly filtros = this.fb.group({
    clienteId: this.fb.control<number | null>(null),
    estado: this.fb.control<EstadoVenta | null>(null),
    desde: this.fb.control<string | null>(null),
    hasta: this.fb.control<string | null>(null),
  });

  protected readonly ahora = signal(new Date());

  /** Suma de los totales de la página mostrada (solo ventas REGISTRADAS). */
  protected readonly totalPagina = computed(() =>
    (this.resultado()?.contenido ?? [])
      .filter(v => v.estado === 'REGISTRADA')
      .reduce((suma, v) => suma + v.total, 0));

  /** Texto con los filtros aplicados, para el encabezado impreso. */
  protected resumenFiltros(): string {
    const f = this.filtros.getRawValue();
    const cliente = this.clientes().find(c => c.id === f.clienteId);
    const partes = [
      cliente ? `cliente ${cliente.apellidos}, ${cliente.nombres}` : '',
      f.estado ? `estado ${f.estado}` : '',
      f.desde ? `desde ${f.desde}` : '',
      f.hasta ? `hasta ${f.hasta}` : '',
    ].filter(Boolean);
    return partes.length ? partes.join(' · ') : 'ninguno (todas las ventas)';
  }

  imprimirConsulta(): void {
    this.ahora.set(new Date());
    setTimeout(() => window.print());
  }

  ngOnInit(): void {
    this.clienteService.listar(0, 100).subscribe({
      next: p => this.clientes.set(p.contenido),
      error: (err: HttpErrorResponse) => this.error.set(mensajeError(err)),
    });
    this.cargar();
  }
```

```
  buscar(): void {
    const { desde, hasta } = this.filtros.getRawValue();
    if (desde && hasta && desde > hasta) {
      this.error.set('La fecha «desde» no puede ser posterior a la fecha «hasta».');
      return;
    }
    this.pagina.set(0);
    this.cargar();
  }

  limpiar(): void {
    this.filtros.reset();
    this.buscar();
  }

  irA(pagina: number): void {
    this.pagina.set(pagina);
    this.cargar();
  }

  private cargar(): void {
    const filtro: FiltroVentas = this.filtros.getRawValue();
    this.cargando.set(true);
    this.error.set(null);
    this.ventaService.buscar(filtro, this.pagina(), 10).subscribe({
      next: r => {
        this.resultado.set(r);
        this.cargando.set(false);
      },
      error: (err: HttpErrorResponse) => {
        this.error.set(mensajeError(err));
        this.cargando.set(false);
      },
    });
  }
}
```

Archivo: `src/app/features/ventas/pages/venta-list/venta-list.html`

```
<section class="page">
  <header class="page-header">
    <h2>Consulta de ventas</h2>
    <div class="botones no-imprimir">
      <button type="button" class="btn" [disabled]="!resultado()?.contenido?.length"
(click)="imprimirConsulta()">🖶 Imprimir consulta</button>
      <a routerLink="nueva" class="btn btn-primary">+ Nueva venta</a>
    </div>
  </header>

  <!-- Solo en el papel: qué filtros produjeron este listado -->
  <p class="solo-impresion resumen-filtros">
    Filtros: {{ resumenFiltros() }} · Impreso el {{ ahora() | date: 'dd/MM/yyyy HH:mm' }}
  </p>

  <form [formGroup]="filtros" (ngSubmit)="buscar()" class="card filtros no-imprimir">
    <label>
      Cliente
      <select class="input" formControlName="clienteId">
        <option [ngValue]="null">Todos</option>
        @for (c of clientes(); track c.id) {
          <option [ngValue]="c.id">{{ c.apellidos }}, {{ c.nombres }}</option>
        }
      </select>
    </label>
    <label>
      Estado
      <select class="input" formControlName="estado">
        <option [ngValue]="null">Todos</option>
        <option value="REGISTRADA">Registrada</option>
        <option value="ANULADA">Anulada</option>
      </select>
    </label>
    <label>Desde <input class="input" type="date" formControlName="desde" /></label>
    <label>Hasta <input class="input" type="date" formControlName="hasta" /></label>
    <div class="botones">
      <button type="button" class="btn" (click)="limpiar()">Limpiar</button>
```

```
      <button type="submit" class="btn btn-primary">Buscar</button>
    </div>
  </form>

  @if (error()) {
    <div class="alerta">{{ error() }}</div>
  }

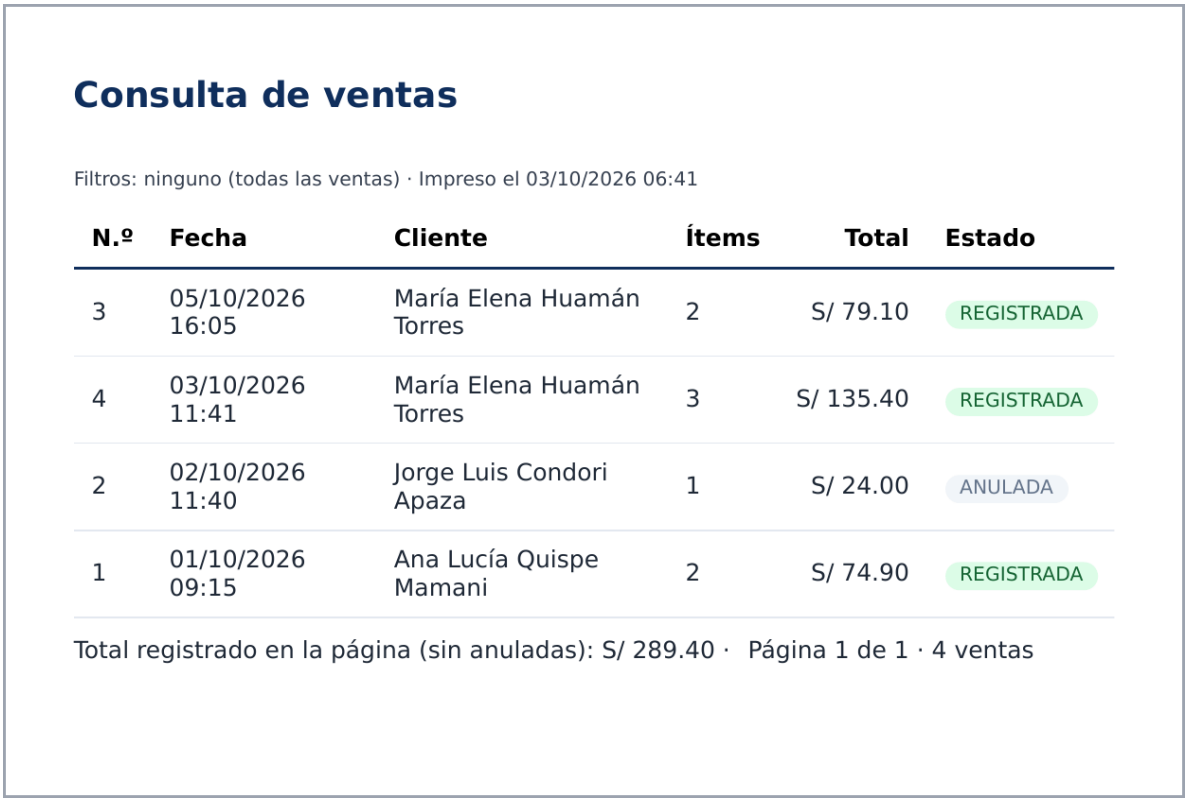
  @if (cargando()) {
    <p>Buscando ventas…</p>
  } @else if (resultado(); as r) {
    <table class="tabla">
      <thead>
        <tr><th>N.º</th><th>Fecha</th><th>Cliente</th><th>Ítems</th><th
class="monto">Total</th><th>Estado</th><th class="no-imprimir"></th></tr>
      </thead>
      <tbody>
        @for (v of r.contenido; track v.id) {
          <tr>
            <td>{{ v.id }}</td>
            <td>{{ v.fecha | date: 'dd/MM/yyyy HH:mm' }}</td>
            <td>{{ v.clienteNombre }}</td>
            <td>{{ v.detalles.length }}</td>
            <td class="monto">{{ v.total | currency: 'PEN' : 'S/ ' }}</td>
            <td><span class="badge" [class.inactivo]="v.estado === 'ANULADA'">{{ v.estado
}}</span></td>
            <td class="no-imprimir acciones-fila">
              <a [routerLink]="[v.id]" class="btn">Ver</a>
              <a [routerLink]="[v.id]" [queryParams]="{ imprimir: 1 }" class="btn" title="Imprimir
comprobante" aria-label="Imprimir comprobante">🖶</a>
            </td>
          </tr>
        } @empty {
          <tr><td colspan="7" class="vacio">No hay ventas con esos filtros.</td></tr>
        }
      </tbody>
    </table>
    <footer class="paginador">
      <span class="solo-impresion">Total registrado en la página (sin anuladas): {{ totalPagina() |
currency: 'PEN' : 'S/ ' }} · </span>
      <span>Página {{ r.pagina + 1 }} de {{ r.totalPaginas || 1 }} · {{ r.totalElementos }}
ventas</span>
      <button type="button" class="btn no-imprimir" [disabled]="r.pagina === 0"
(click)="irA(r.pagina - 1)">Anterior</button>
      <button type="button" class="btn no-imprimir" [disabled]="r.ultima" (click)="irA(r.pagina +
1)">Siguiente</button>
    </footer>
  }
</section>
```

Archivo: `src/app/features/ventas/pages/venta-list/venta-list.css`

```
.filtros { display: grid; grid-template-columns: 2fr 1fr 1fr 1fr auto; gap: 12px; align-items: end;
margin-bottom: 16px; }
.filtros label { display: flex; flex-direction: column; gap: 4px; font-size: 14px; font-weight:
600; }
.botones { display: flex; gap: 8px; }
.monto { text-align: right; white-space: nowrap; }
.vacio { text-align: center; color: #6b7280; padding: 24px; }
.paginador { display: flex; align-items: center; justify-content: flex-end; gap: 12px; margin-top:
12px; }
.page-header .botones { align-items: center; }
.acciones-fila { display: flex; gap: 6px; }
.resumen-filtros { margin: 0 0 12px; font-size: 13px; color: #374151; }
@media print { .paginador { justify-content: flex-start; } .paginador .solo-impresion { display:
inline; } }
```



Figura 10. Consulta de ventas filtrada por estado y rango de fechas. Nota. Elaboración propia.



Consulta de ventas

Filtros: ninguno (todas las ventas) · Impreso el 03/10/2026 06:41

N.º	Fecha	Cliente	Ítems	Total	Estado
3	05/10/2026 16:05	María Elena Huamán Torres	2	S/ 79.10	REGISTRADA
4	03/10/2026 11:41	María Elena Huamán Torres	3	S/ 135.40	REGISTRADA
2	02/10/2026 11:40	Jorge Luis Condori Apaza	1	S/ 24.00	ANULADA
1	01/10/2026 09:15	Ana Lucía Quispe Mamani	2	S/ 74.90	REGISTRADA

Total registrado en la página (sin anuladas): S/ 289.40 · Página 1 de 1 · 4 ventas

Figura 11. Consulta de ventas impresa con el resumen de filtros. Nota. Elaboración propia.

Paso 10. Probar la operación

Con PharmaBackend y `ng serve` en ejecución, abre la pestaña **Red** (F12) y ejecuta estas pruebas. Guarda una captura de cada una.

N.º	Acción	Resultado esperado
1	Pulsar «Revisar venta» con el formulario vacío	Mensajes en cliente y en la línea; no aparece la confirmación.
2	Buscar cliente escribiendo parte del apellido sin tilde y luego un texto que no existe	Aparece el cliente; con el texto inexistente, «Ningún cliente activo coincide…». Los inactivos nunca aparecen.
3	Elegir el cliente, buscar un producto, agregarlo y poner cantidad 3	Se crea la línea; subtotal = precio × 3 y total actualizado al instante.
4	Agregar dos productos más con Enter y quitar uno con ✕	Enter agrega el primer resultado sin enviar el formulario; subtotales y total se recalculan.
5	Buscar y agregar otra vez un producto que ya está en la venta	El resultado indica «En la venta: n»; al agregarlo sube su cantidad y no se crea otra línea.
6	Escribir una cantidad mayor que el stock	Mensaje «Supera el stock» en la línea; «Revisar venta» no avanza.
7	Corregir y pulsar «Revisar venta»	Aparece el resumen con cliente, número de productos y total estimado.
8	Pulsar «Volver»	Se cierra el resumen sin enviar ninguna petición.
9	Revisar otra vez y pulsar «Confirmar venta»	POST 201; navega al comprobante con «Venta registrada correctamente».
10	Comparar el total del comprobante con el estimado	Coinciden (el servidor usa los mismos precios).
11	Con la casilla «Imprimir el comprobante al registrar» marcada, confirmar una venta	Se abre el diálogo de impresión; la vista previa no muestra menú ni botones e incluye el DNI y el pie.
12	En la consulta, pulsar 🖶 en una venta anulada	Se abre el comprobante con el sello «ANULADA» y el diálogo de impresión.
13	Filtrar la consulta y pulsar «Imprimir consulta»	La vista previa muestra el resumen de filtros, la tabla sin botones y el total registrado de la página.
14	En Productos, verificar el stock de lo vendido	El stock bajó en las cantidades vendidas.
15	Forzar stock insuficiente: abrir dos pestañas y vender en ambas las últimas unidades	La segunda recibe 409 con el mensaje de stock; no se registra nada.
16	Consultar ventas filtrando por cliente y por fechas	GET a `/ventas/buscar` solo con los filtros llenos; aparece la venta registrada.
17	Poner «desde» posterior a «hasta» y buscar	Mensaje en la SPA; no se envía la petición.

Paso 11. Versionar el trabajo

```
git add .
git commit -m "feat(ventas): formulario cabecera-detalle con líneas dinámicas"
git commit -am "feat(ventas): calcular subtotales y total con signals"
git commit -am "feat(ventas): buscadores de cliente y producto, validar stock y confirmar"
git commit -am "feat(ventas): comprobante imprimible y consulta de ventas con filtros"
git push -u origin feature/ventas-<apellido>
```

Haz cada commit cuando esa parte funcione y abre una solicitud de incorporación hacia `develop`. Después de crear archivos nuevos usa `git add .` antes del commit.

Producto esperado

- Rama `feature/ventas-<apellido>` con al menos cuatro commits que sigan la convención y una solicitud de incorporación hacia `develop`.
- Módulo Ventas integrado al menú: formulario cabecera–detalle con cálculos, validaciones y confirmación; comprobante; consulta paginada con filtros.
- Documento PDF `Apellido_LP2_S9_Practica.pdf` con las capturas de las diecisiete pruebas del paso 10.

Lista de cotejo

N.º	Criterio observable	Sí	No
1	El feature Ventas tiene modelos, servicio, tres páginas y rutas con carga diferida, y aparece en el menú.		
2	El cliente y los productos se eligen con buscadores que filtran por nombre, DNI o categoría sin distinguir tildes y avisan cuando no hay coincidencias.		
3	El formulario usa un FormArray para el detalle; las líneas se agregan desde el buscador y se pueden quitar.		
4	Cada fila está conectada con `formArrayName` y `[formGroupName]`.		
5	Subtotales y total se calculan con `computed` a partir de `toSignal(valueChanges)` y se redondean a dos decimales.		
6	El request solo envía `clienteId`, `productoId` y `cantidad`; nunca precios ni totales.		
7	Se validan cliente obligatorio, al menos una línea, cantidad entera mayor que cero, stock por línea; agregar un producto ya elegido aumenta su cantidad en vez de repetirlo.		
8	Antes del POST se muestra un resumen y el usuario debe confirmar; «Volver» no envía nada.		
9	Los errores 400, 404 y 409 del backend se muestran con su mensaje y el usuario puede corregir.		
10	El comprobante muestra la venta guardada con el total calculado por el servidor.		
11	El comprobante y la consulta se imprimen sin menú ni botones; se puede imprimir al registrar y desde cada fila de la consulta.		
12	La consulta envía solo los filtros con valor, valida el rango de fechas y está paginada.		
13	La rama tiene al menos cuatro commits según la convención y una solicitud de incorporación hacia `develop`.		

3. Problemas frecuentes

Síntoma	Causa probable	Solución
«Cannot find control with path: detalles -> 0 -> productoId»	Falta `formArrayName="detalles"` o `[formGroupName]="i"` en la plantilla.	Coloca `formArrayName` en el contenedor de las filas y `[formGroupName]="i"` en cada fila.
El total no cambia al escribir la cantidad	Los cálculos no leen un signal del formulario.	Usa `toSignal(this.form.valueChanges, { initialValue: this.form.value })` y `computed`.
El total muestra 37.499999	Suma de decimales en coma flotante.	Redondea cada subtotal y el total con `redondear()`.
El POST responde 400 en `detalles[0].cantidad`	La cantidad viaja como texto o vacía.	Convierte con `Number()` al armar el `VentaRequest` y valida `min(1)`.
El buscador de clientes nunca muestra resultados	Se trató la respuesta paginada de `/clientes` como arreglo.	Usa `clientes.contenido` y pide `tamanio` 100.
«huaman» no encuentra a «Huamán»	Se comparó con `includes()` sin normalizar.	Usa `coincide()`, que quita tildes y pasa a minúsculas.
Enter en el buscador abre la confirmación	El Enter envía el formulario de la venta.	Maneja `(keydown.enter)` y llama a `evento.preventDefault()`, como en `agregarPrimero()`.
El formulario se envía sin productos	Se usó `Validators.minLength(1)` con un FormArray vacío; ese validador no marca error con arreglos vacíos.	Usa `Validators.required` en el FormArray.
Al imprimir sale el menú lateral	La regla `@media print` para el layout se puso en `styles.css` y pierde frente a los estilos encapsulados del componente.	Coloca la regla en `main-layout.css`, como en el paso 8.
La vista previa sale sin colores de fondo	El navegador omite fondos al imprimir.	Usa `print-color-adjust: exact` o activa «Gráficos de fondo» en el diálogo.
La consulta responde 409 «El rango de fechas es inválido»	La fecha «desde» es posterior a «hasta».	Valida el rango en la SPA antes de buscar, como en `buscar()`.
Después del 409 de stock el formulario queda bloqueado	No se reinició `guardando` o `confirmando` en el `error`.	En el `error` del POST pon ambos signals en `false` y muestra el mensaje.

4. Referencias

Angular. (s. f.-a). *FormArray*. https://angular.dev/api/forms/FormArray

Angular. (s. f.-b). *Reactive forms*. https://angular.dev/guide/forms/reactive-forms

Angular. (s. f.-c). *RxJS interop with Angular signals*. https://angular.dev/ecosystem/rxjs-interop

Bampakos, A., & Deeleman, P. (2023). *Learning Angular* (4.ª ed.). Packt Publishing.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.